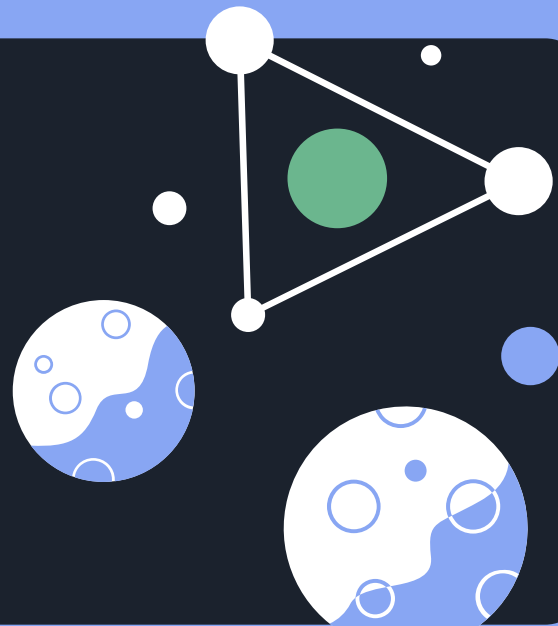


# Figgy Stacked



Sara Bouftas, Namish Kaistha, Aditi Ram,  
Omar Sharaf, Francis Velasco





# Agenda

0

Tech Stack

1

Design Flow +  
Architecture

2

Technical  
Challenges

3

Demo

4

Reflection

## Front End

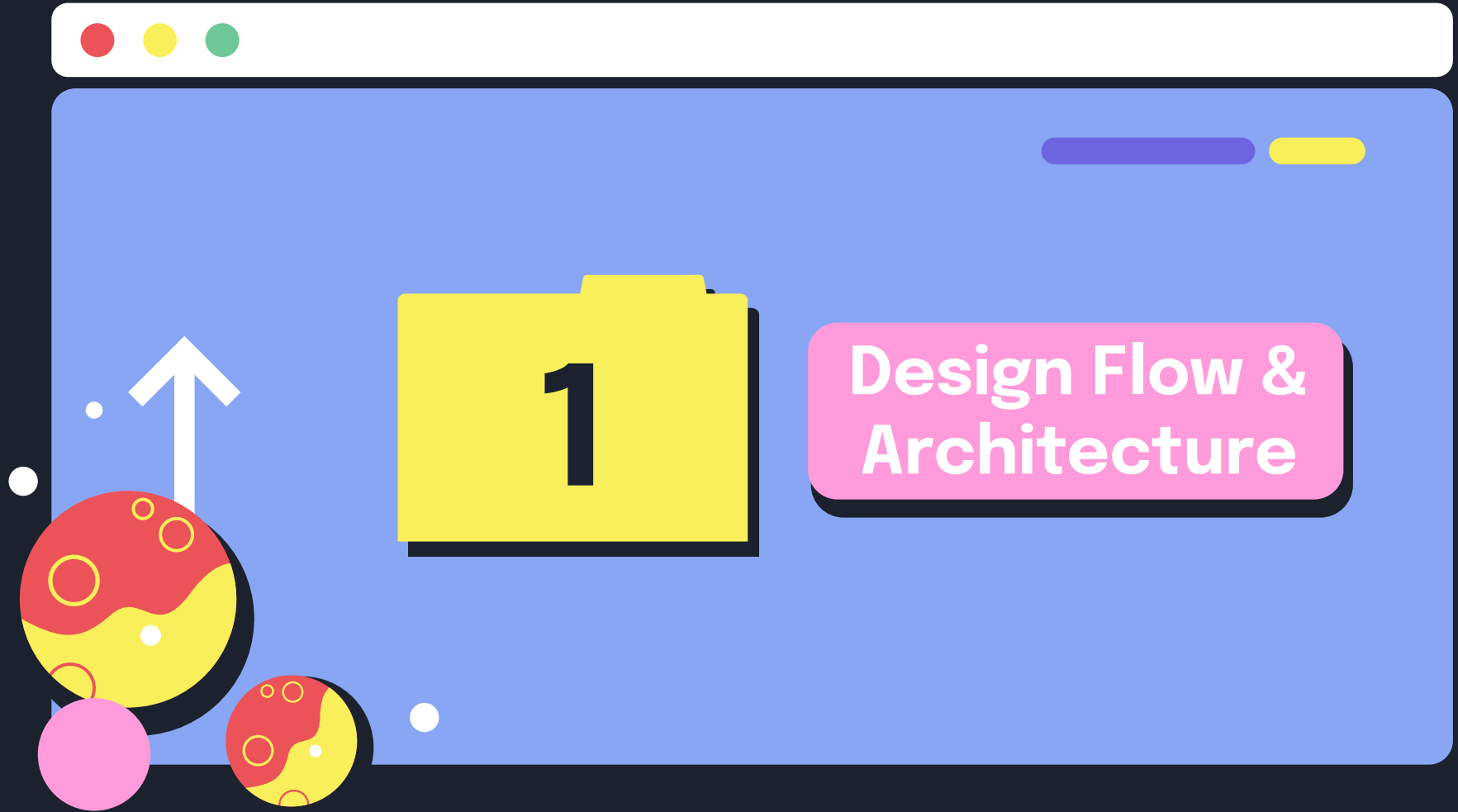


React Native, TypeScript  
styled with StyleSheet

## Back End

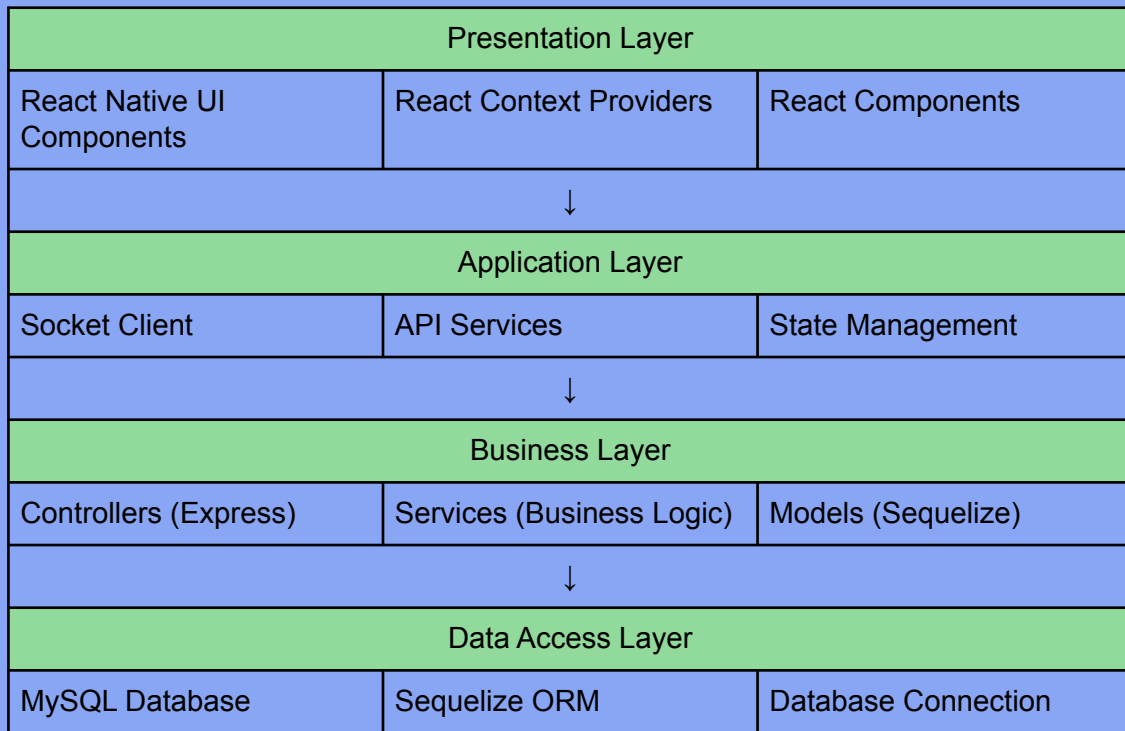


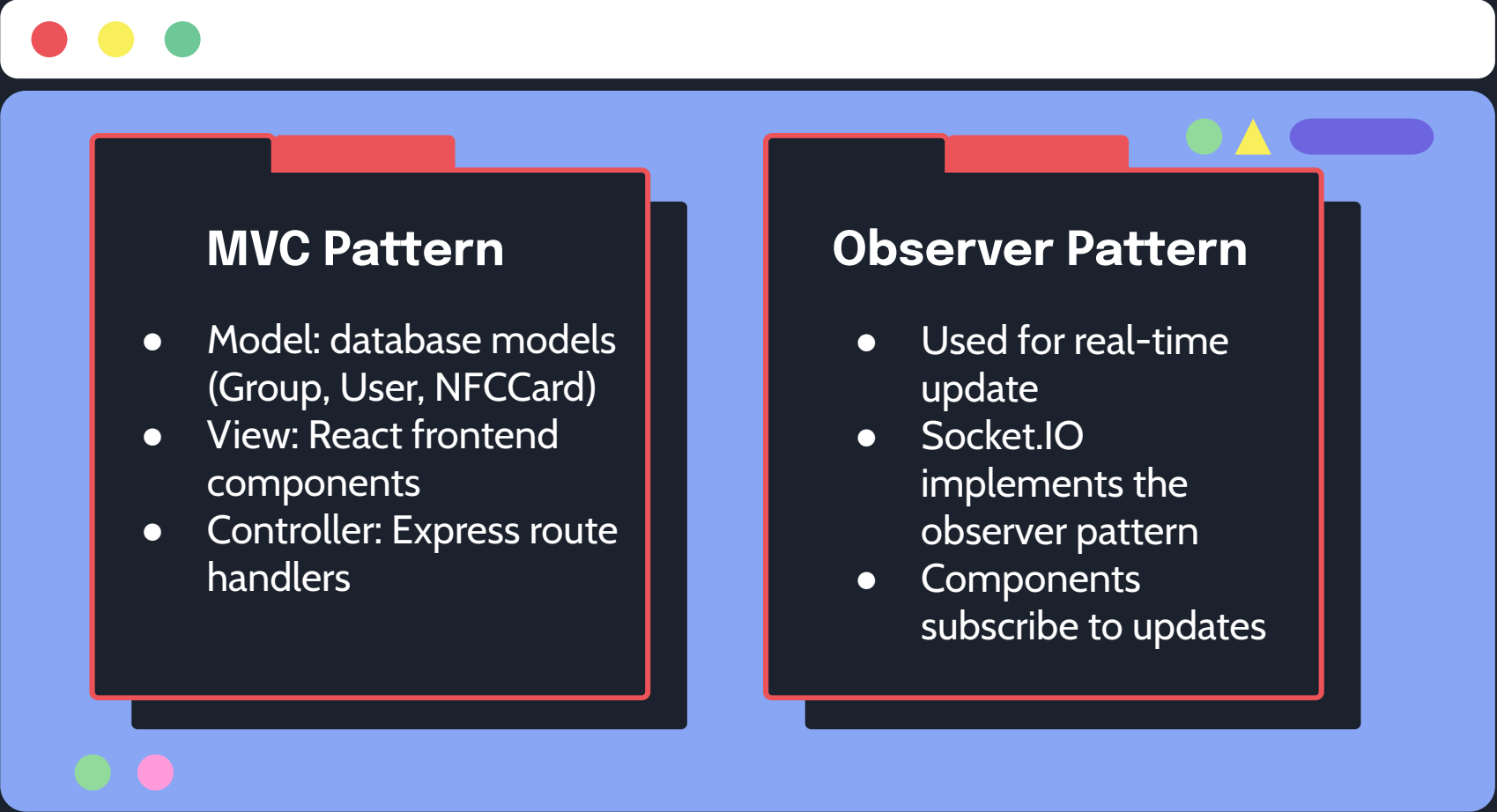
node.js, express.js,  
sequelize, mysql, socket.io



## Layered Architecture

- All components are part of one application
- Clear separations of layers
  - Routes (API endpoints)
  - Controllers
  - Models
  - Database





## MVC Pattern

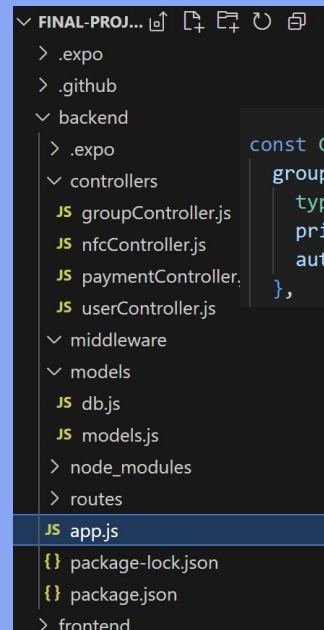
- Model: database models (Group, User, NFCCard)
- View: React frontend components
- Controller: Express route handlers

## Observer Pattern

- Used for real-time update
- Socket.IO implements the observer pattern
- Components subscribe to updates

# Backend Server Organization (Express.js)

- Organized our codebase into three main sections:
  - Groups (for managing group payments)
  - Users (for user accounts)
  - NFC (for card management)
- We separated our routing logic, business logic, and database access in our file structure.



```
const Group = sequelize.define('genGroup', {  
  groupId: {  
    type: DataTypes.INTEGER,  
    primaryKey: true,  
    autoIncrement: true,  
  },  
});
```

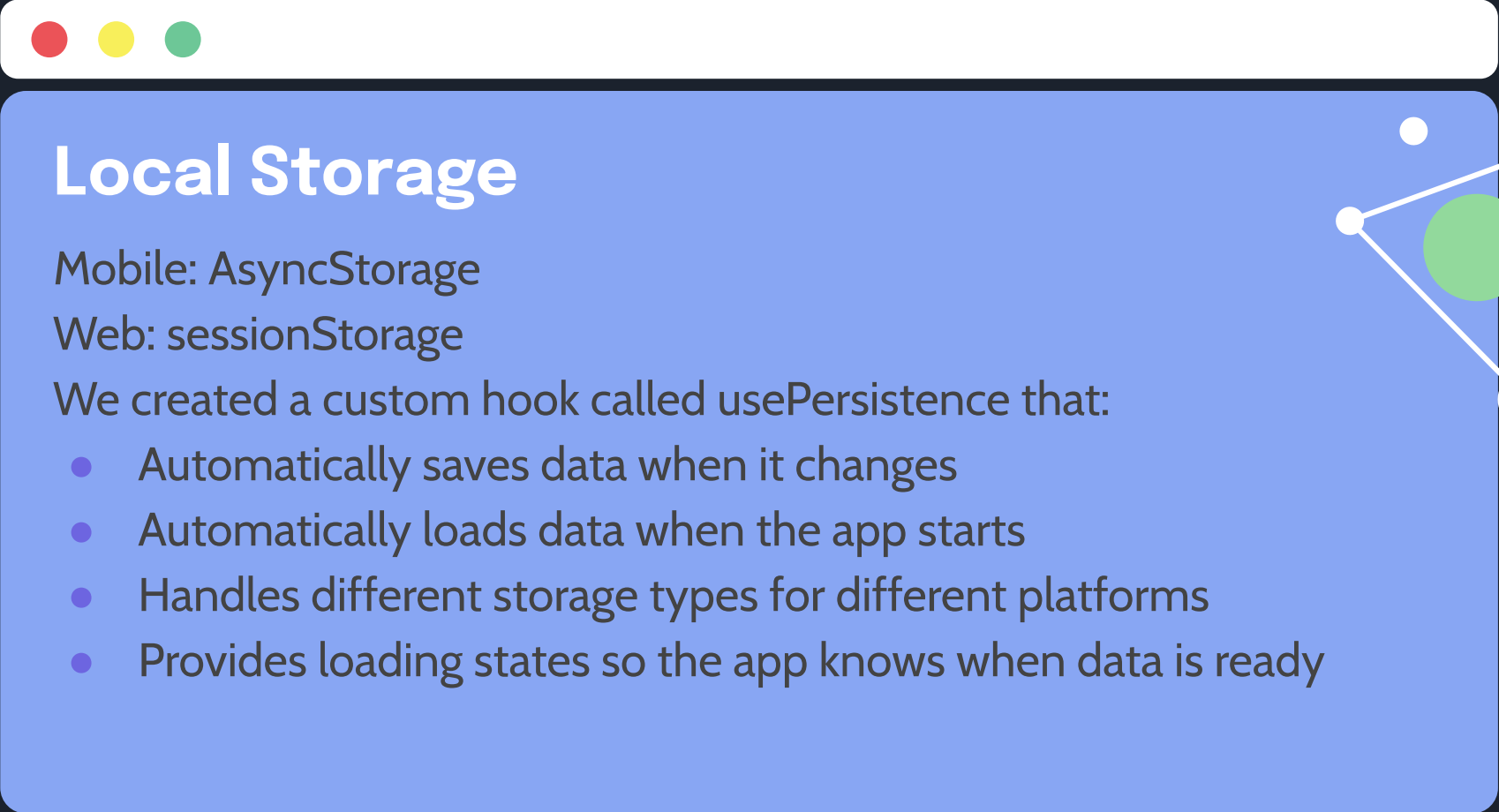
# State Management

React Context API - built in feature that lets you share state/context across multiple components without having to drill props (dependency injection)

- Components only re-render when the specific context values they use actually change
- Two custom hooks (useUser and useGroup) that provide access to our global state
  - Ex. Our payment confirmation page just calls useUser and useGroup hooks (no prop drilling)

```
<App>
  <HomePage userId="123" groupId="456">
    <GroupList userId="123" groupId="456">
      <GroupItem userId="123" groupId="456">
        <Member userId="123" groupId="456" />
      </GroupItem>
    </GroupList>
  </HomePage>
</App>
```

```
<UserProvider>
  <GroupProvider>
    <App>
      <HomePage>
        <GroupList>
          <GroupItem>
            <Member />
          </GroupItem>
        </GroupList>
      </HomePage>
    </App>
  </GroupProvider>
</UserProvider>
```

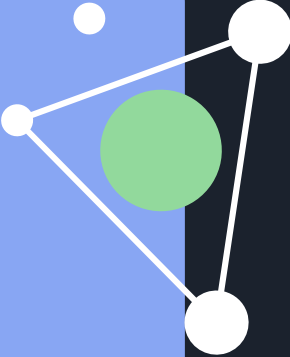


# Local Storage

Mobile: AsyncStorage

Web: sessionStorage

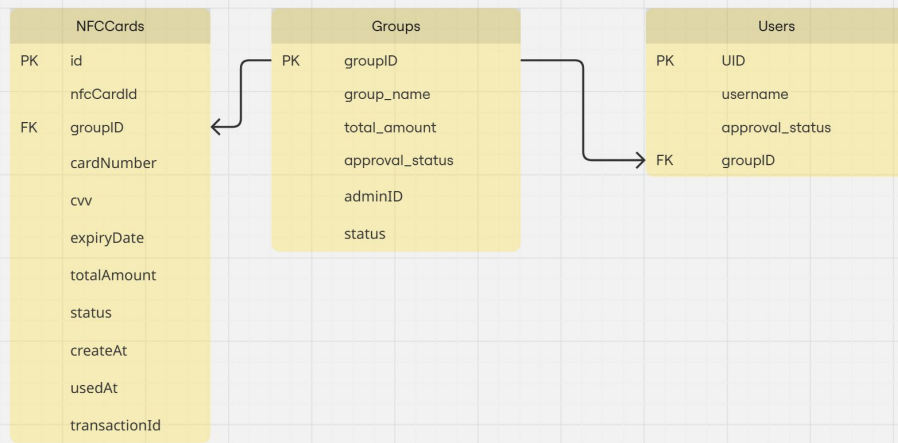
We created a custom hook called usePersistence that:

- Automatically saves data when it changes
  - Automatically loads data when the app starts
  - Handles different storage types for different platforms
  - Provides loading states so the app knows when data is ready
- 

# Database Setup MySQL

- Use MySQL to store all our data
- 3 main types of data:
  - Groups (payment groups)
  - Users (people using the app)
  - NFC Cards (payment cards)

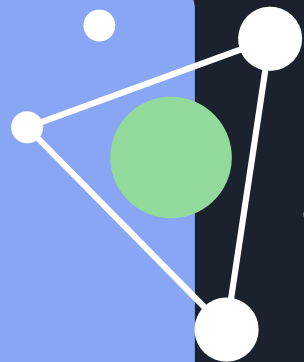
## paymentapp Tables





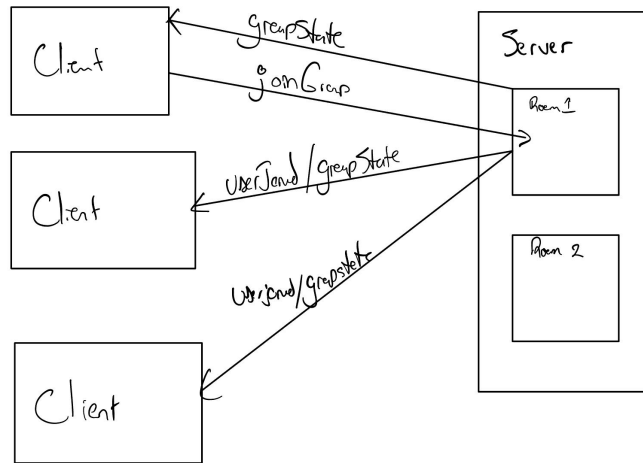
# Real-time Features (Socket.io)

- Websockets for real time
- Using socket.io rooms to group clients by roomId
- Client → Server events
  - joinGroup
  - userApproval
- Server → Client events
  - groupState, allApproved, paymentComplete
- Event Driven Architecture

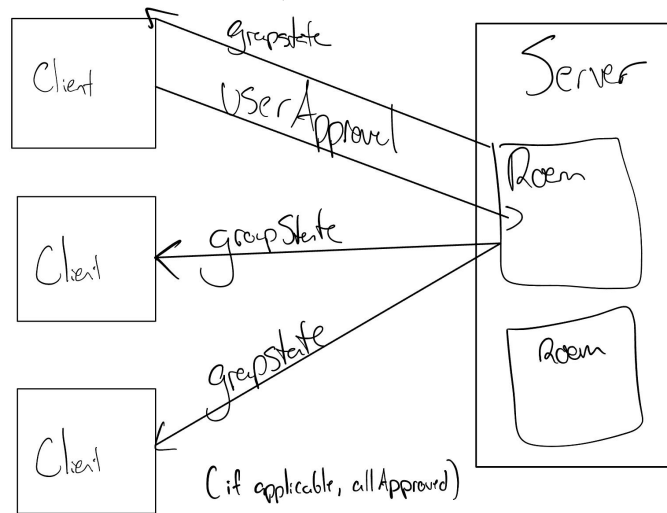


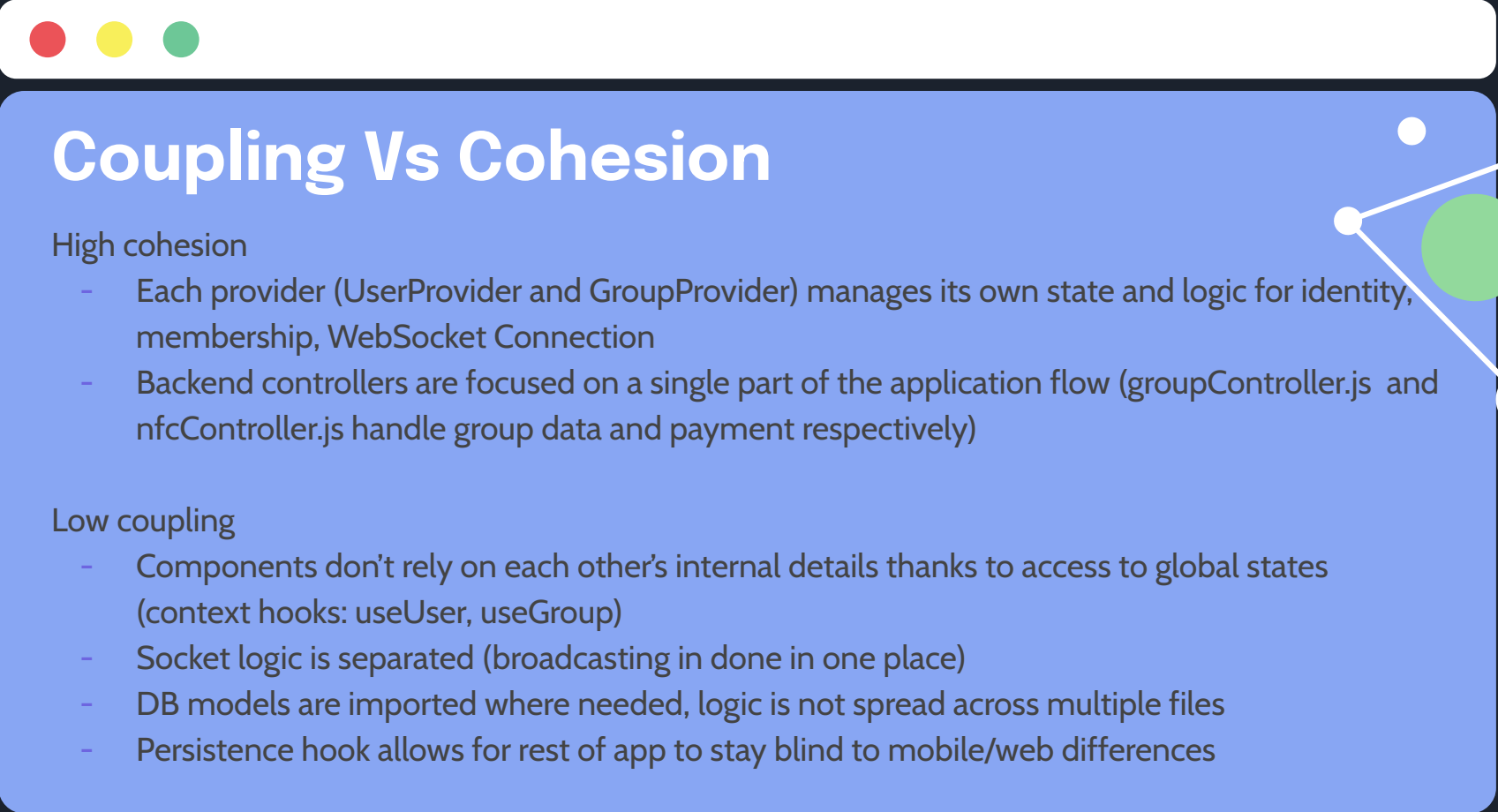


### Join group flow



### Approval flow





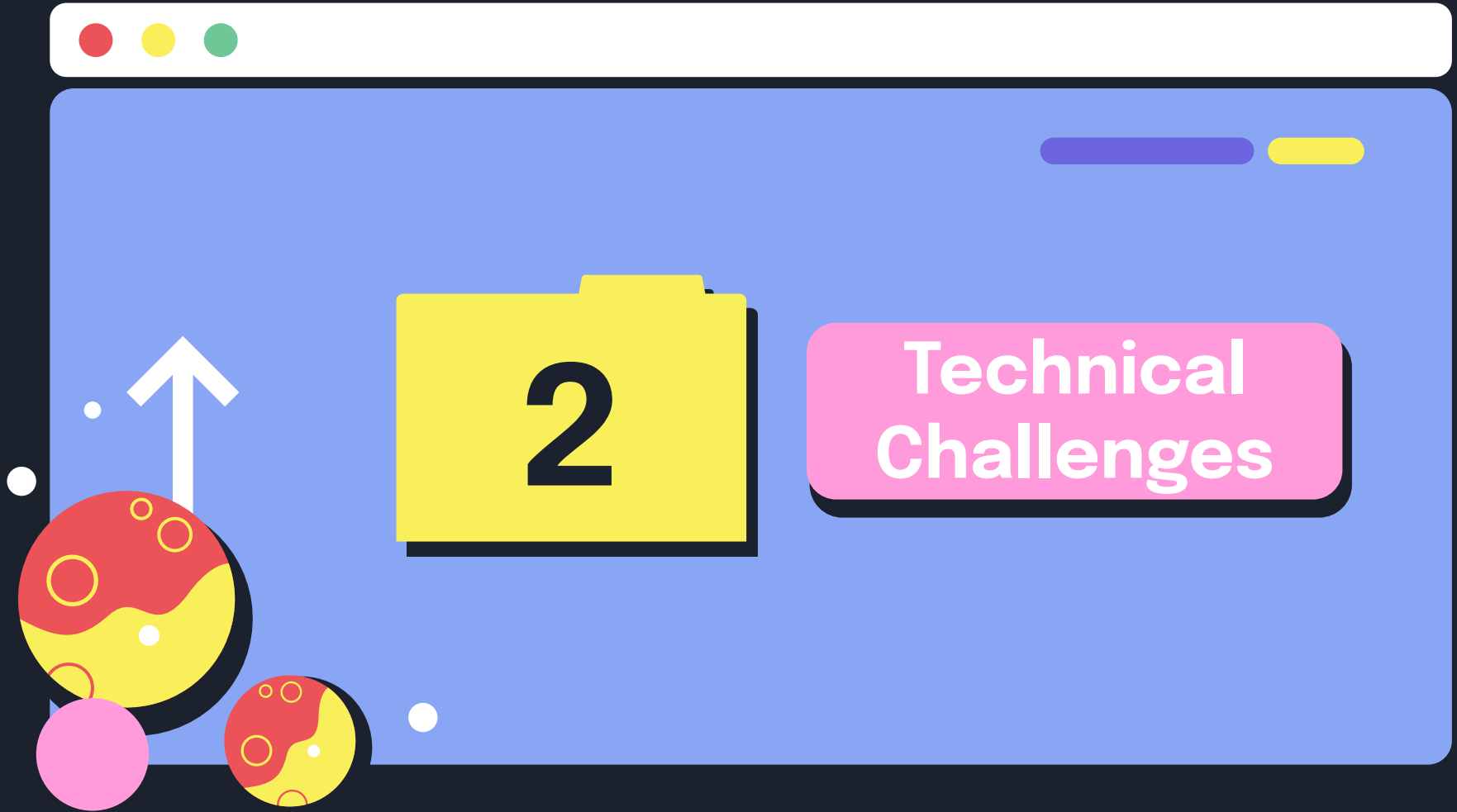
# Coupling Vs Cohesion

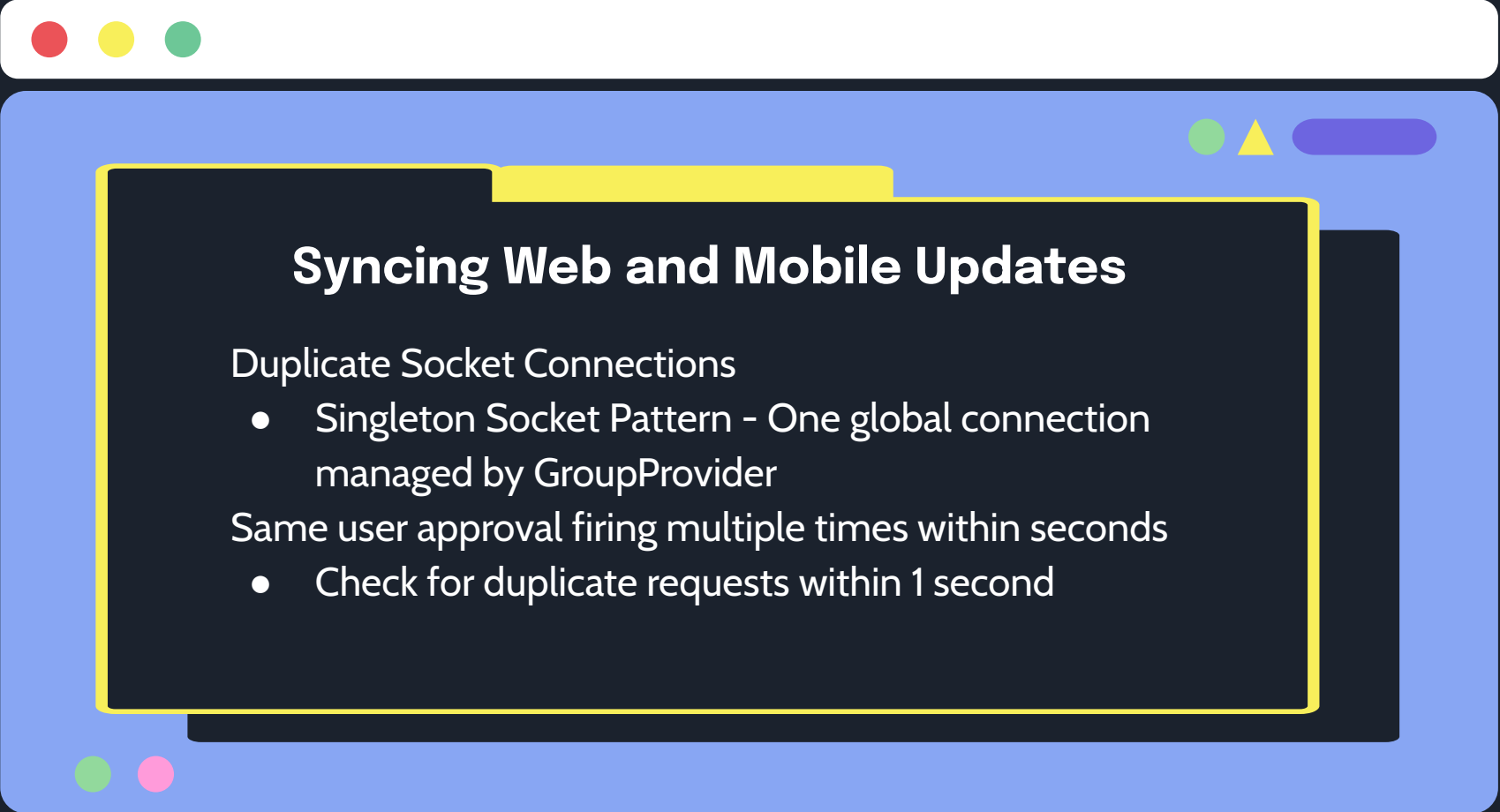
## High cohesion

- Each provider (UserProvider and GroupProvider) manages its own state and logic for identity, membership, WebSocket Connection
- Backend controllers are focused on a single part of the application flow (groupController.js and nfcController.js handle group data and payment respectively)

## Low coupling

- Components don't rely on each other's internal details thanks to access to global states (context hooks: useUser, useGroup)
- Socket logic is separated (broadcasting is done in one place)
- DB models are imported where needed, logic is not spread across multiple files
- Persistence hook allows for rest of app to stay blind to mobile/web differences





## Syncing Web and Mobile Updates

Duplicate Socket Connections

- Singleton Socket Pattern - One global connection managed by GroupProvider

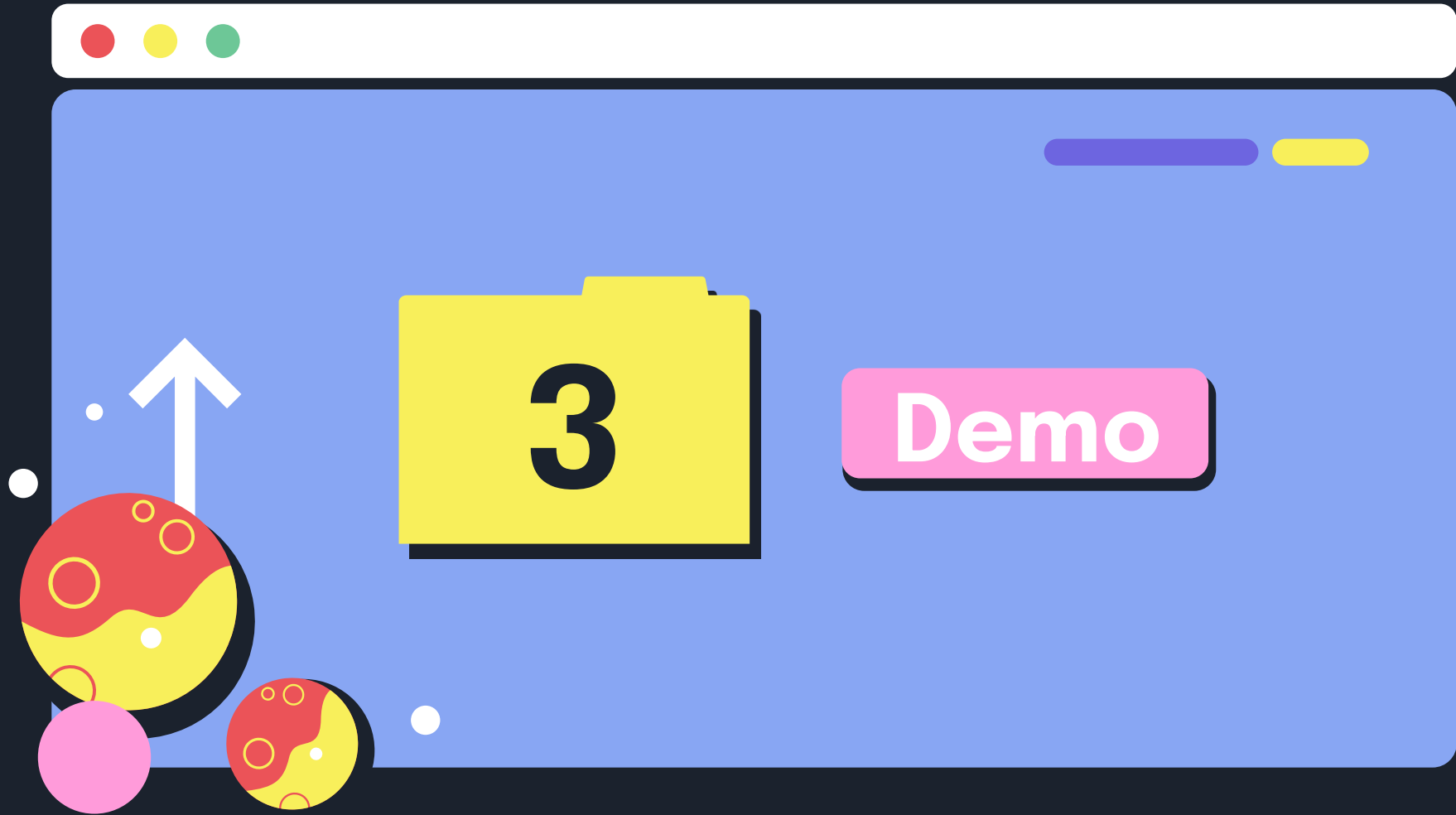
Same user approval firing multiple times within seconds

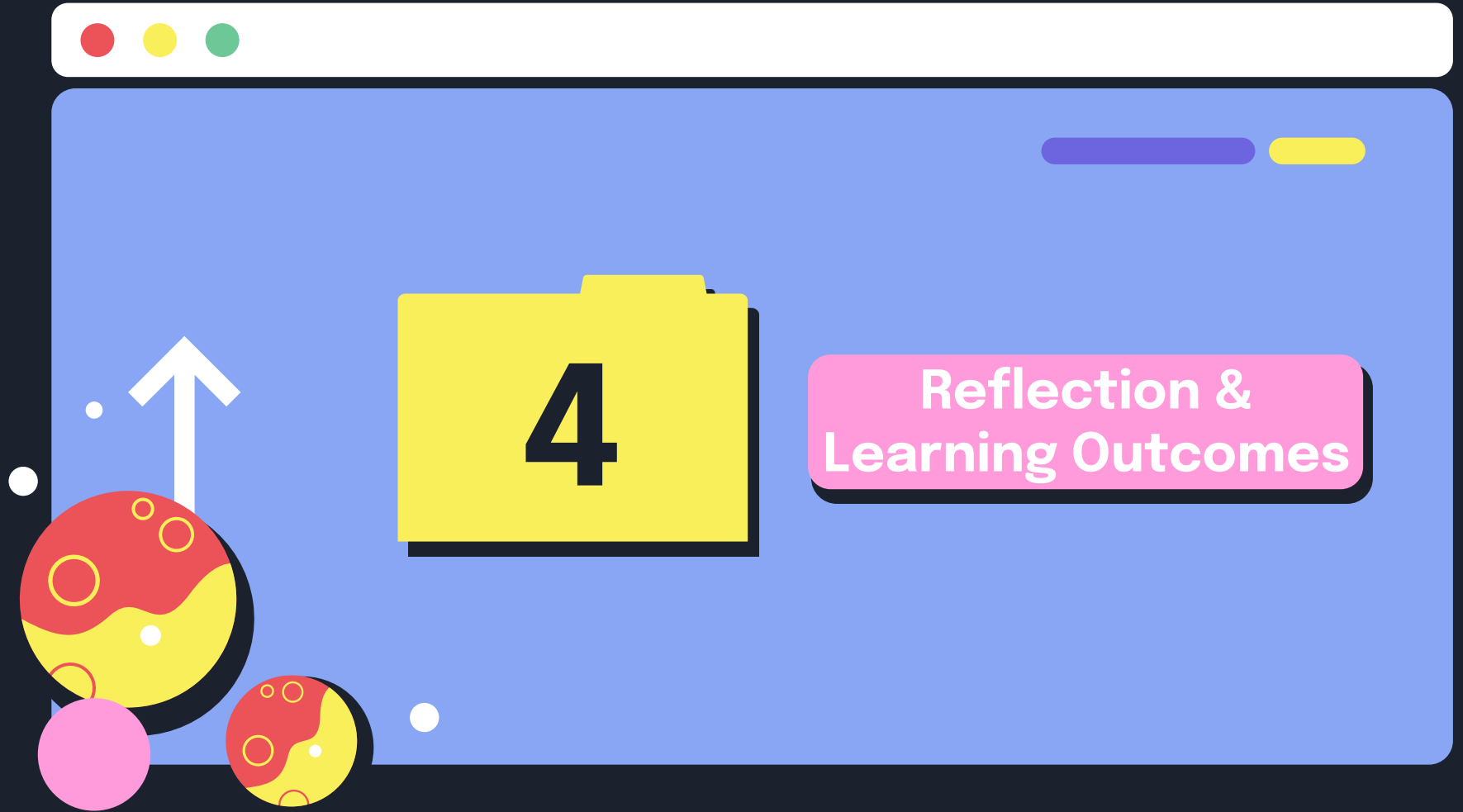
- Check for duplicate requests within 1 second

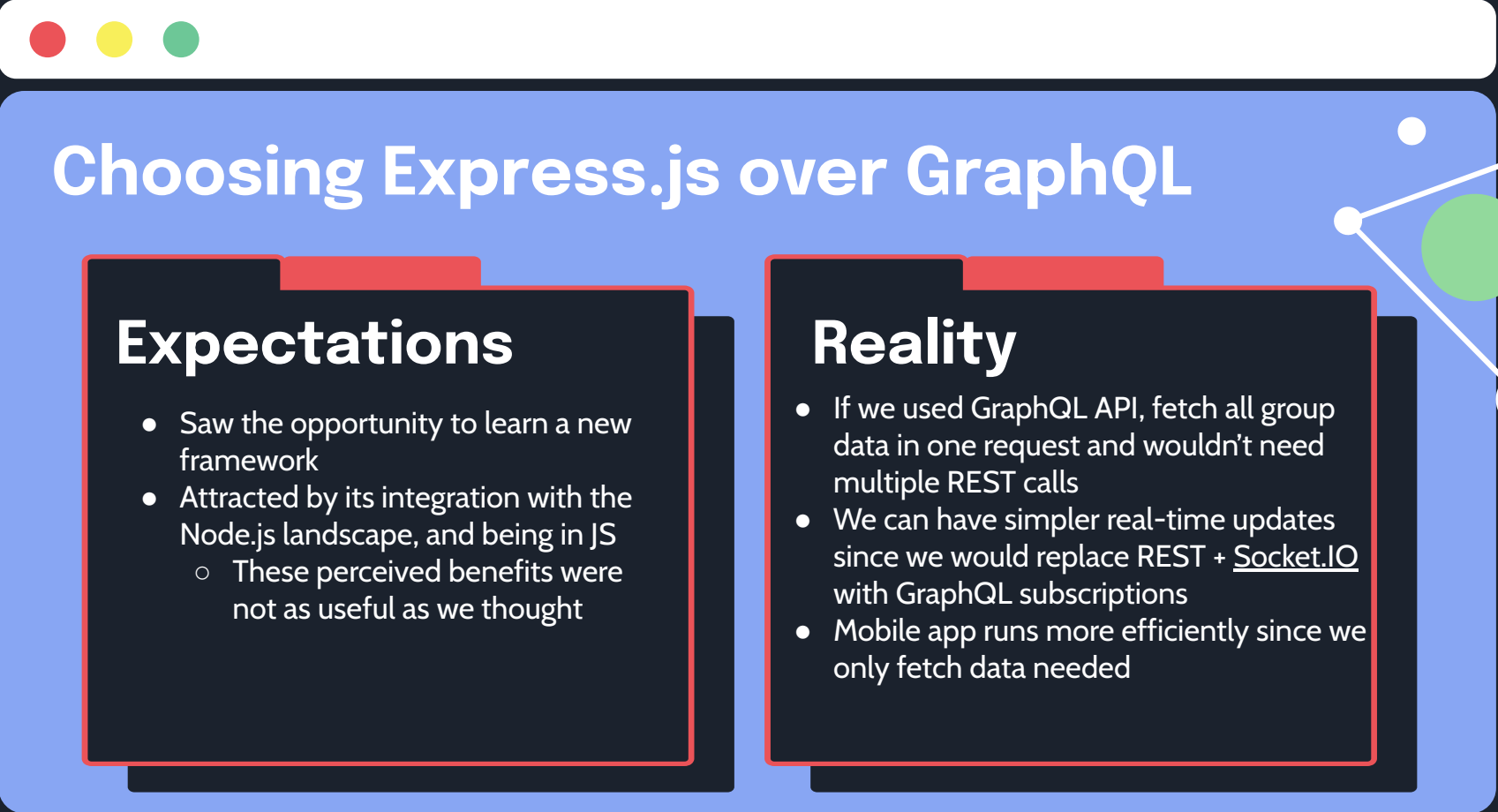
## Syncing Web and Mobile → Sockets

Certain users could not view changes on mobile

- Broadcasted group state to ALL users







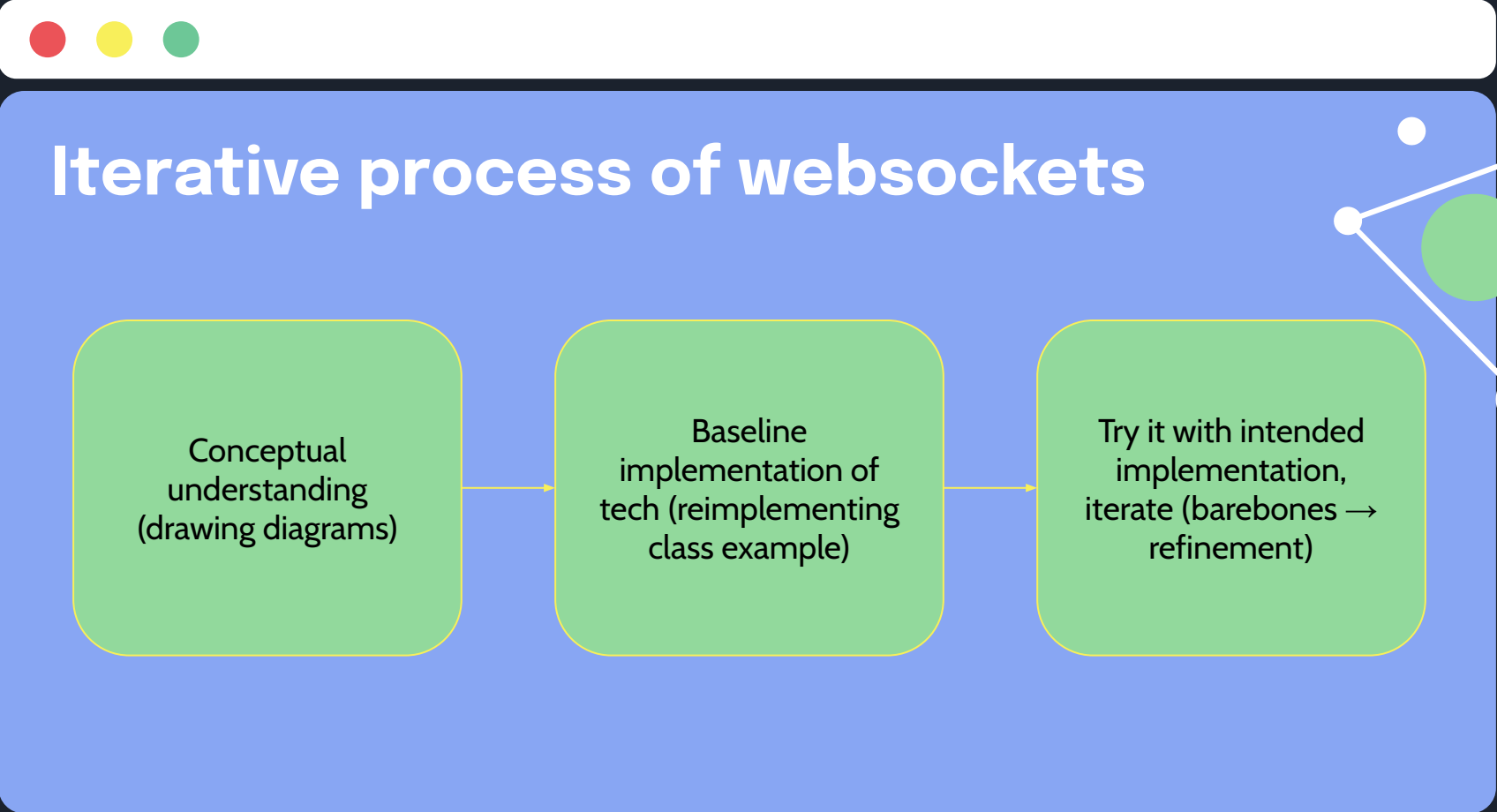
# Choosing Express.js over GraphQL

## Expectations

- Saw the opportunity to learn a new framework
- Attracted by its integration with the Node.js landscape, and being in JS
  - These perceived benefits were not as useful as we thought

## Reality

- If we used GraphQL API, fetch all group data in one request and wouldn't need multiple REST calls
- We can have simpler real-time updates since we would replace REST + Socket.IO with GraphQL subscriptions
- Mobile app runs more efficiently since we only fetch data needed

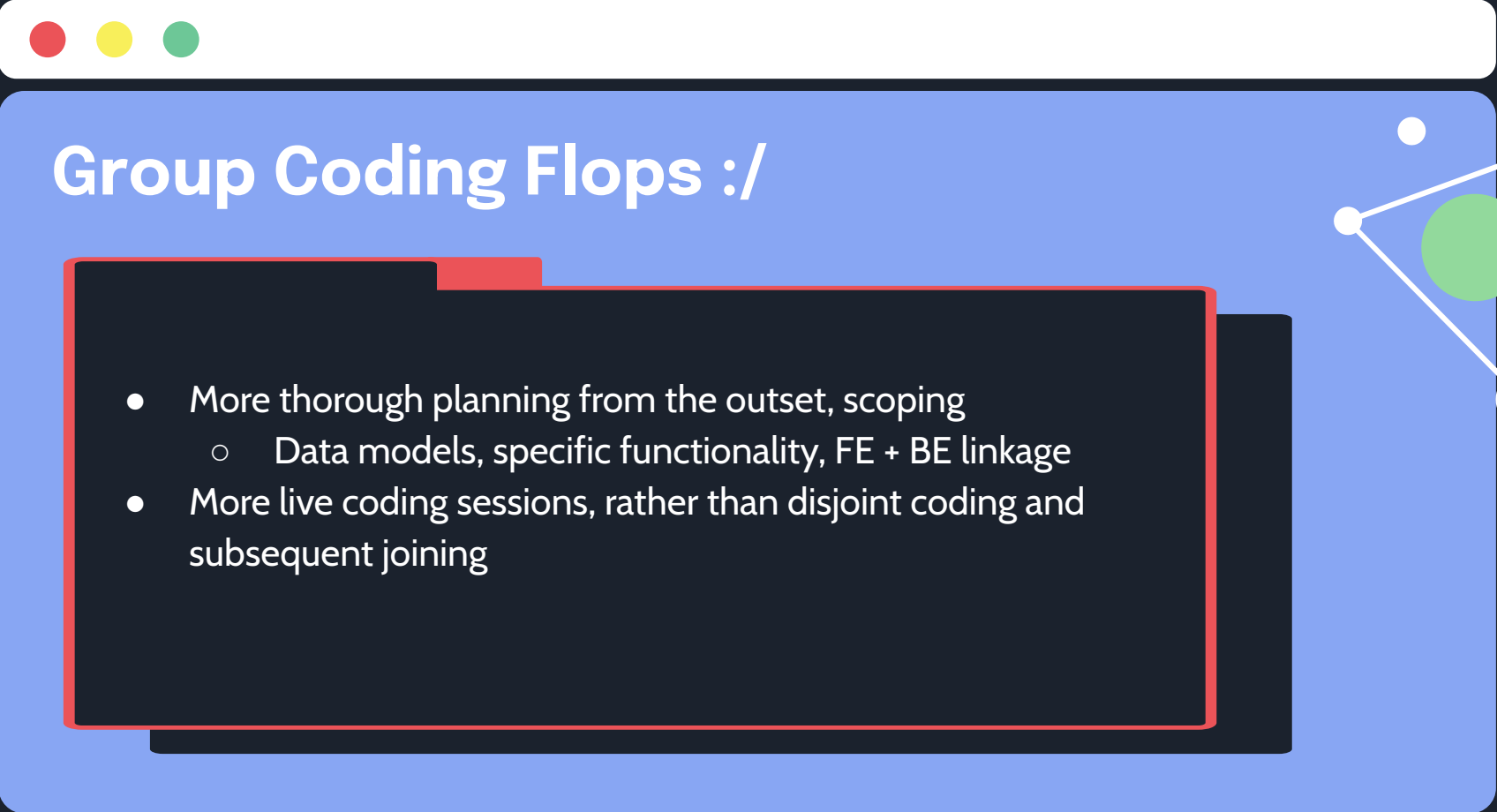


# Iterative process of websockets

Conceptual  
understanding  
(drawing diagrams)

Baseline  
implementation of  
tech (reimplementing  
class example)

Try it with intended  
implementation,  
iterate (barebones →  
refinement)



# Group Coding Flops :/

- More thorough planning from the outset, scoping
  - Data models, specific functionality, FE + BE linkage
- More live coding sessions, rather than disjoint coding and subsequent joining



# Future Iterations



**Authentication**



**Sending Money to  
Leader Before  
Payment**

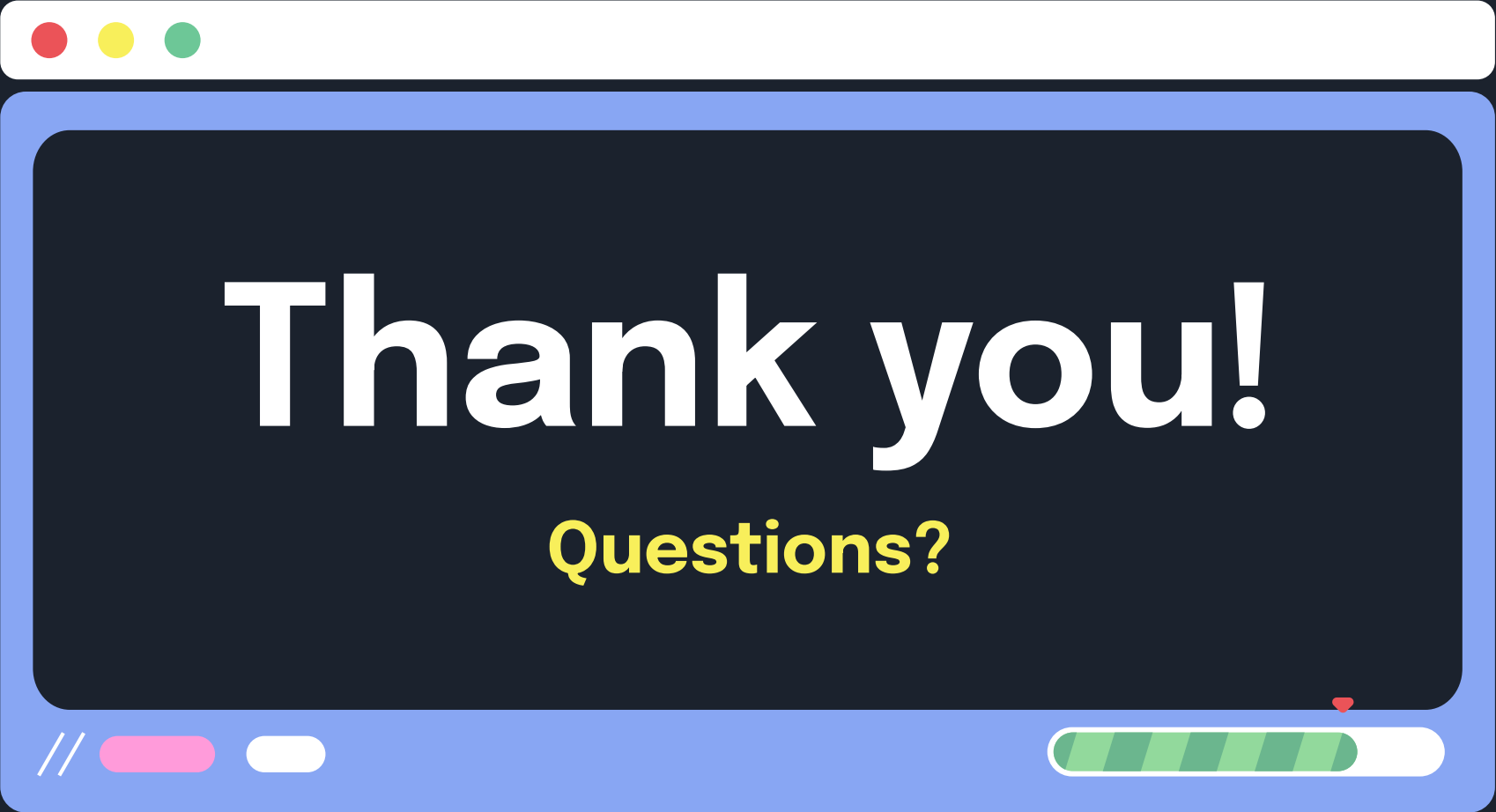


**OCR on a  
Receipt**



**Manual  
Splitting**





# Thank you!

Questions?